ЛР1. Реалізація та перевірка зворотного поширення похибки

Мережа: X → Linear(4, 8) → ReLU → Linear(8, 3).
Втрата – крос-ентропія, усереднена за об'єктами.

Дані: Iris, стратифікований поділ 35/15 на клас, стандартизація за статистиками навчальної вибірки (ddof = 0).

Команди запуску

```
cd lab1
uv sync
uv run lab1_layers.py                    # правильна реалізація
uv run lab1_layers.py --experiment       # дослід із навмисною помилкою (за замовчуванням працює правильна реалізація)
```

Перелік використаних формул

Афінний шар у пакетній формі

$$Z = A \cdot W + b$$
$$A \in \mathbb{R}^{N \times n_{in}}, \quad W \in \mathbb{R}^{n_{in} \times n_{out}}, \quad b \in \mathbb{R}^{n_{out}}, \quad Z \in \mathbb{R}^{N \times n_{out}}$$

- A — вхід шару (для першого шару $A = X$); кожен рядок — окремий об'єкт пакета.
- W — матриця ваг, b — вектор зсувів; ті самі W і b застосовуються до всіх рядків.
- Зсув додається за правилом broadcasting: (N, m) + (m,) → (N, m), без фізичного копіювання b.

Функція активації ReLU

$$\mathrm{ReLU}(z) = \max(0, z)$$
$$\mathrm{ReLU}'(z) = \begin{cases} 1, & z > 0 \\ 0, & z < 0 \end{cases}$$
$$D = \mathrm{diag}\big(\varphi'(z_1), \dots, \varphi'(z_m)\big)$$

- z — передактивація (елемент Z1); φ застосовується поелементно, тому якобіан діагональний, і зворотний прохід через неї коштує $\mathcal{O}(m)$.

- У нулі ReLU не диференційовна (субградієнт лежить у [0,1]); PyTorch у зворотному проході бере 0. Маска ($Z1 > 0$) у коді в точці $z = 0$ дає 0, тобто збігається з PyTorch.

Softmax

$$q_k = \frac{\exp(z_k)}{\sum_j \exp(z_j)}, \quad k = 1, \dots, K$$

- $z \in \mathbb{R}^K$ — логіти (тут $K = 3$);
- q_k — ймовірність класу k: усі q_k додатні, сума дорівнює 1. Логіти самі по собі не є ймовірностями.

Чисельно стабільний log-sum-exp та log-softmax

$$m = \max_j z_j$$

$$\log \sum_j \exp(z_j) = m + \log \sum_j \exp(z_j - m)$$

$$\log q_k = z_k - \log \sum_j \exp(z_j) = (z_k - m) - \log \sum_j \exp(z_j - m)$$

- Тотожність точна. Після зсуву всі $z_j - m \leq 0$, тож exp ≤ 1, а принаймні один доданок дорівнює 1 — сума не може стати нулем. Без зсуву exp переповнюється вже при $z \approx 88.7$ у float32.

Крос-ентропія та середня втрата

$$H(p, q) = -\sum_k p_k \log q_k$$

Для one-hot p:

$$H(p, q) = -\log q_y = -z_y + \log \sum_j \exp(z_j)$$

$$L = \frac{1}{N} \sum_i (-\log P\,[i, y_i])$$

- p — цільовий розподіл (one-hot), q — розподіл моделі. $N = 105$. PyTorch cross_entropy очікує логіти, а не ймовірності й сама об'єднує log-softmax і NLL.

Градієнт втрати за логітами

Для одного об'єкта:

$$\frac{\partial L_i}{\partial z_k} = q_k - \mathbb{1}[k = y_i]$$

Для середньої втрати:

$$dZ2 = \frac{P - Y}{N}$$

- Y — one-hot матриця міток. Множник $1/N$ виникає тому, що L — середнє за N об'єктами.
-

Ланцюгове правило та добуток вектора на якобіан (VJP)

$$J_{g\circ f}(x) = J_g\big(f(x)\big) \cdot J_f(x)$$

$$\nabla_x L^T = \frac{\partial L}{\partial Z^{(L)}} \cdot J_L \cdot J_{L-1} \cdot \ldots \cdot J_1$$

$$\text{VJP: } v^T J, \quad [J]_{ij} = \frac{\partial f_i}{\partial x_j}$$

- На практиці повні якобіани не будують: обчислюють лише дію $v^T J$, що коштує приблизно стільки ж, скільки сам прямий прохід.

Градієнти афінного шару та поширення похибки назад

$$dW = A^T \cdot dZ \quad (n_{in} \times n_{out})$$

$$db = \sum_i d\,Z[i,:] \quad (n_{out},)$$

$$dA = dZ \cdot W^T \quad (N \times n_{in})$$

$$dZ1 = dA1 \odot \mathbb{1}[Z1 > 0] \quad (N \times 8)$$

- Для зворотного проходу зберігаються активації прямого проходу (A для dW), передактивація або маска ($Z1$ для ReLU) і ваги ($W2$ для dA).

Чисельна похідна (центральна різниця)

$$f(x+h) = f(x) + f'(x)h + \frac{1}{2}f''(x)h^2 + o(h^2)$$

$$g_{num} = \frac{L(\theta+\varepsilon) - L(\theta-\varepsilon)}{2\varepsilon}, \quad \varepsilon = 10^{-6}$$

- Значення $\varepsilon = 10^{-6}$ задано завданням. Занадто малий крок у скінченній арифметиці дає втрату точності від округлення, тому використовується float64.

Критерії проходження перевірок

- Звірка з PyTorch: $|a - b| \le 10^{-12}$

- Чисельна перевірка: $|g_{num} - g_{manual}| \leq 10^{-7}$

Навмисна помилка: пропущене ділення на N

$$L_{mean} = \frac{1}{N}\sum_i L_i\,, \quad L_{sum} = \sum_i L_i = N \cdot L_{mean}$$

$$dZ2_{bug} = P - Y = N \cdot dZ2$$

- Без множника $1/N$ backward обчислює градієнт суми втрат, а forward лишається середньою втратою, тож градієнти перебільшуються рівно в $N = 105$ разів.

Дані та ініціалізація

$$x_{std} = \frac{x - \mu_{train}}{\sigma_{train}} \quad \text{(рахується з ddof} = 0)$$

$$W1 \sim \mathcal{N}\left(0, \sqrt{\frac{2}{4}}\right), \quad W2 \sim \mathcal{N}\left(0, \sqrt{\frac{2}{8+3}}\right), \quad b1 = 0, \quad b2 = 0$$

Формули зворотного проходу та розмірності

Ініціалізація

Параметр	Розмірність	Ініціалізація
W1	(4, 8)	He: $\mathcal{N}\left(0, \sqrt{2/4}\right)$
b1	(8,)	нулі
W2	(8, 3)	Xavier: $\mathcal{N}\left(0, \sqrt{2/11}\right)$
b2	(3,)	нулі

Прямий прохід і стабільна втрата

$$Z1 = X \cdot W1 + b1$$

$$A1 = \max(0, Z1)$$

$$Z2 = A1 \cdot W2 + b2$$

$$m_i = \max_k Z\,2[i, k]$$

$$\log P\,[i,k] = (Z2[i,k] - m_i) - \log \sum_j \exp(Z2[i,j] - m_i)$$

$$L = -\frac{1}{N} \sum_i \log P\,[i, y_i]$$

Зворотний прохід ($N = 105$)

$$dZ2 = \frac{P - Y}{N}$$

$$dW2 = A1^T \cdot dZ2$$

$$db2 = \sum_i d\,Z2[i,:]$$

$$dA1 = dZ2 \cdot W2^T$$

$$dZ1 = dA1 \odot \mathbb{1}[Z1 > 0]$$

$$dW1 = X^T \cdot dZ1$$

$$db1 = \sum_i d\,Z1[i,:]$$

Розмірності проміжних масивів

Масив	Розмірність	Масив	Розмірність
X	(105, 4)	dA1	(105, 8)
Z1	(105, 8)	dZ1	(105, 8)
A1	(105, 8)	dW1	(4, 8)
Z2	(105, 3)	db1	(8,)
P, Y, dZ2	(105, 3)	dW2	(8, 3)
y	(105,)	db2	(3,)

Збережені проміжні значення

- **X** — потрібен для $dW1 = X^T \cdot dZ1$.
- **Z1** — потрібен для маски ReLU $\mathbb{1}[Z1 > 0]$.
- **A1** — потрібен для $dW2 = A1^T \cdot dZ2$.
- **P** та **y** — потрібні для початкового сигналу похибки $dZ2 = (P - Y)/N$.
- **W2** — потрібна для поширення похибки назад: $dA1 = dZ2 \cdot W2^T$.

2. Звірка з PyTorch

Аналогічну мережу створено в PyTorch (cross_entropy з усередненням). Параметри скопійовано з NumPy-мережі; torch.nn.Linear зберігає ваги як (out, in), тому копіюються $W1^T$ та $W2^T$, а градієнти ваг перед порівнянням транспонуються назад.

Критерій для кожної пари: $|a - b| \leq$ 1e-12, усі значення скінченні.

Значення втрати:

NumPy = 1.4562007801142816;
PyTorch = 1.456200780114282

Величина	Макс. абсолютна різниця NumPy / PyTorch	Перевірку пройдено
Втрата	2.220e-16	Так
Градієнт W1	2.776e-17	Так
Градієнт b1	3.643e-17	Так
Градієнт W2	1.110e-16	Так
Градієнт b2	1.388e-16	Так

3. Чисельна перевірка чотирьох градієнтів

Центральна різниця з ε = 1e-6: g_num = (L(θ+ε) − L(θ−ε)) / (2ε). Для кожного параметра значення зберігається, збурюється на +ε і −ε, потім відновлюється. Початкові ваги, повна навчальна вибірка. Критерій: $|g_num - g_manual| \leq$ 1e-7.

Параметр	Градієнт backward()	Чисельна похідна	Абс. різниця	Пройдено
W1[0, 0]	-1.9920184331e-02	-1.9920184391e-02	6.034e-11	Так
b1[0]	-7.0566111584e-02	-7.0566111621e-02	3.633e-11	Так
W2[0, 0]	1.3523020861e-01	1.3523020870e-01	9.218e-11	Так
b2[0]	8.5521644842e-02	8.5521644988e-02	1.460e-10	Так

4. Дослід із навмисною помилкою

Помилка: у градієнті за логітами прибрано ділення на N ($dZ2 = P - Y$ замість$(P - Y)/N$).

Функція втрат, еталон PyTorch і формула чисельної похідної не змінені; ваги між порівняннями не оновлювалися.

Прогноз

Звірка з еталоном PyTorch покаже, що значення втрати збігається, але зафіксує помилку в усіх обчислених градієнтах.

Чисельна перевірка знайде розбіжність для всіх ненульових параметрів, оскільки ручне значення буде в 105 разів більшим за еталонне чисельне. Але для параметрів із нульовим градієнтом перевірка пройде успішно суто формально — система побачить збіг незважаючи на помилку в коді.

Результати

Втрата NumPy до і після помилки однакова: 1.4562007801142816.

Відношення градієнтів «з помилкою / правильний» для всіх ненульових елементів дорівнює 105.

W1: 32/32 ненульових
b1: 8/8,
W2: 24/24,

b2: 3/3).

Звірка з PyTorch (з помилкою):

Величина	Макс. абсолютна різниця NumPy / PyTorch	Перевірку пройдено
Втрата	2.220e-16	очікується: Так
Градієнт W1	1.852e+01	очікується: Ні
Градієнт b1	1.564e+01	очікується: Ні
Градієнт W2	3.419e+01	очікується: Ні
Градієнт b2	1.120e+01	очікується: Ні

Чисельна перевірка (з помилкою):

Параметр	Градієнт backward()	Чисельна похідна	Абс. різниця	Пройдено
W1[0, 0]	-2.0916193547e+00	-1.9920184391e-02	2.072e+00	Ні
b1[0]	-7.4094417164e+00	-7.0566111621e-02	7.339e+00	Ні
W2[0, 0]	1.4199171904e+01	1.3523020870e-01	1.406e+01	Ні
b2[0]	8.9797727084e+00	8.5521644988e-02	8.894e+00	Ні

Прогноз підтвердився, адже прямий прохід усе ще рахує середню втрату, а backward без 1/N відповідає градієнту суми, тобто $L_{sum} = N \cdot L_{mean}$. Тому помилку не видно за значенням втрати – її виявляють лише перевірки градієнтів.

Висновки

Написаний алгоритм зворотного поширення похибки на NumPy дає ідентичні результати як із вбудованим автоматичним диференціюванням у PyTorch, так і з методом чисельної перевірки.

Щоб зворотний прохід працював коректно, потрібно кешувати проміжні значення прямого проходу: вхідні дані X, передактивацію Z1, активацію A1, фінальні ймовірності P (разом із мітками y) та матрицю ваг W2.

Сама по собі чисельна перевірка є ненадійною. Вона тестує лише вибрані параметри і може не помітити помилку, якщо градієнт параметра дорівнює нулю. Тому її необхідно використовувати лише в парі з еталонною моделлю (PyTorch), яка звіряє абсолютно всі елементи матриць.

Практичний дослід довів, що пропущене ділення на розмір вибірки ніяк не впливає на значення функції втрат, але при цьому всі ненульові градієнти збільшуються у 105 разів.

Використані джерела

- Формули було взято з Лекції 1 та Лекції 2.

- Код і текст цього звіту підготовлено з використанням генеративного інструмента Claude.